SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	YUVARAJ R	**Reg. No.:**	192524058
Section / Batch:	2025	**Date:**	

Code of Conduct

*I **YUVARAJ R / 192524058** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Yuvaraj R

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Annexure A – Coding Challenge

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

1. Study and understand the assigned rule.
2. Explain the linguistic purpose of the rule.
3. Describe the conditions under which the rule should be applied.
4. Implement the rule in Python.
5. Prepare at least **10 test words** demonstrating:
    - Words that satisfy the rule.
    - Words that should remain unchanged.
6. Document the input, expected output, and actual output.
7. Explain any limitations or special cases encountered.

Phase 2: Team Integration

As a team, integrate all individual rule implementations into a complete Porter Stemmer.

The team must:

- Arrange the rules in the correct execution order.
- Resolve conflicts between rule implementations.
- Ensure that the output of one rule becomes the input to the next rule.
- Test the complete algorithm using at least **50 different English words**.
- Compare the team's results with the NLTK Porter Stemmer.
- Identify and explain any differences.

Phase 3: Reflection

Prepare a report addressing the following questions:

1. Why is the order of Porter Stemmer rules important?
2. What errors would occur if your assigned rule were skipped?
3. How did integrating multiple rules improve stemming accuracy?
4. Which rule was the most challenging to implement and why?
5. What improvements would you suggest for the Porter Stemming algorithm?

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach

Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Total %
1	15	18	15	20	12	80

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Problem Statement

In Natural Language Processing (NLP), the same root word can occur in different grammatical and derived forms. This may affect applications such as search engines and information retrieval systems, where related forms of a word should be treated similarly.

The Porter Stemming Algorithm is a rule-based stemming technique that reduces English words by applying a sequence of suffix transformation rules. In this assessment, the assigned task is to implement **Step 3 of the Porter Stemmer** using Python.

Step 3 performs further suffix reduction by identifying specific suffixes such as **-icate, -ative, -alize, -iciti, -ical, -ful, and -ness**. The appropriate suffix is removed or replaced only when the stem satisfies the Porter measure condition **m > 0**.

The objective is to design a modular Python implementation that correctly applies the Step 3 rules, preserves words that do not satisfy the required conditions, and demonstrates the implementation using valid and unchanged test cases.

Objective

The main objective is to implement **Step 3 of the Porter Stemming Algorithm** from scratch using Python. The program identifies specific suffixes in an input word and performs the corresponding removal or replacement when the required Porter measure condition **m > 0** is satisfied.

Linguistic Purpose of Step 3

Step 3 is used to perform **further suffix reduction** on words after the earlier stages of the Porter Stemmer have been applied. It mainly handles derivational suffixes that change the form or grammatical category of a word.

The Step 3 rules are:

Suffix	Replacement
-icate	-ic
-ative	Removed
-alize	-al
-iciti	-ic
-ical	-ic
-ful	Removed
-ness	Removed

For example:

triplicate → triplic

formalize → formal

hopeful → hope

goodness → good

These transformations help normalize different derived word forms into simpler stems, which is useful in NLP applications such as **information retrieval, text processing, indexing, and search engines**.

Conditions for Applying Step 3

In the Porter Stemming Algorithm, a Step 3 suffix cannot be removed or replaced simply because it occurs at the end of a word. The transformation is performed only when the stem satisfies the **measure condition** m > 0.

The measure **m** represents the number of **vowel-consonant (VC) sequences** present in the stem.

In this representation:

- **V** represents a vowel: a, e, i, o, u
- **C** represents a consonant
- The letter y is handled separately according to its position in the word.

For example, consider the word hopeful.

- The suffix ful is identified.
- Removing ful gives the stem hope.

The consonant-vowel pattern of hope is:

- h o p e
  C V C V
- The stem contains one complete VC sequence, therefore m = 1.
- Since m > 0, the rule FUL → "" is applied.
- The resulting word is hope.

Therefore:

hopeful → hope

The general condition for Step 3 can be represented as:

(m > 0) suffix → replacement

If no Step 3 suffix is found, or if the stem does not satisfy m > 0, the input word remains unchanged.

Algorithm – Porter Stemmer Step 3

Input: An English word
Output: Word after applying the Porter Stemmer Step 3 rule

Start.

Read the input word and convert it to lowercase.

Define the Step 3 suffix replacement rules:

icate → ic

ative → ""

alize → al

iciti → ic

ical → ic

ful → ""

ness → ""

Check whether the input word ends with any of the Step 3 suffixes.

If a matching suffix is found, remove the suffix temporarily to obtain the stem.

Calculate the Porter measure m of the stem by counting its vowel-consonant (VC) sequences.

Check whether $m > 0$.

If $m > 0$, replace the matched suffix with its corresponding replacement.

If $m = 0$, leave the word unchanged.

If no Step 3 suffix is found, leave the word unchanged.

Display the resulting word.

Stop.

Pseudocode

START

INPUT word

CONVERT word to lowercase

DEFINE Step 3 rules:

ICATE → IC

```
        ATIVE → empty
        ALIZE → AL
        ICITI → IC
        ICAL → IC
        FUL → empty
        NESS → empty
FOR each suffix and replacement
   IF word ends with suffix THEN
      stem = word without suffix
      CALCULATE measure(stem)
      IF measure(stem) > 0 THEN
         word = stem + replacement
      END IF
      BREAK
   END IF
END FOR
OUTPUT word
STOP
```

Python Implementation – Porter Stemmer Step 3

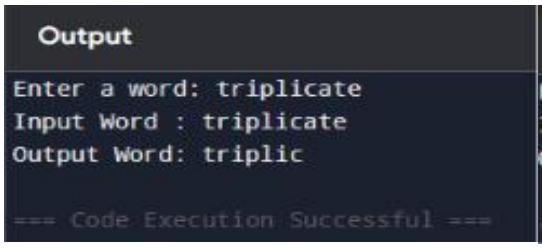
main.py

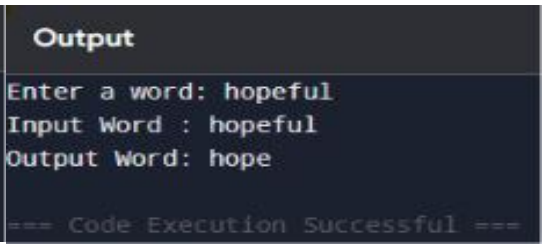
```python
class PorterStemmerStep3:

    # Check whether a character is a consonant
    def is_consonant(self, word, i):

        if word[i] in "aeiou":
            return False

        # Special handling for 'y'
        if word[i] == "y":
            if i == 0:
                return True
            return not self.is_consonant(word, i - 1)

        return True


    # Calculate the Porter measure (m)
    def measure(self, word):

        m = 0
        i = 0
        length = len(word)

        # Skip initial consonants
        while i < length and self.is_consonant(word, i):
            i += 1

        while i < length:

            # Skip vowels
```

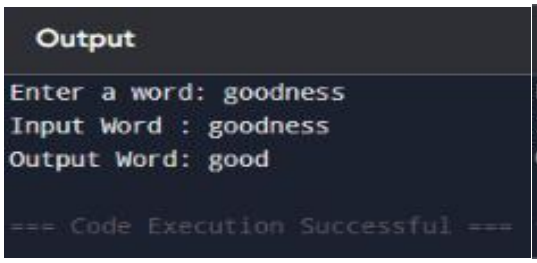
main.py

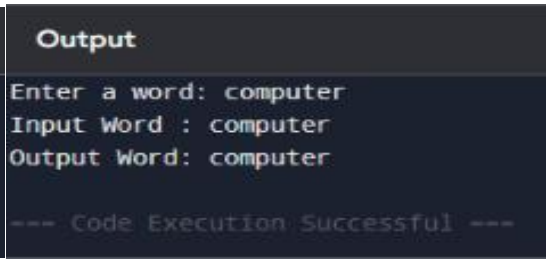
```
30
31              # Skip vowels
32              while i < length and not self.is_consonant(word, i):
33                  i += 1
34
35              # Stop if end of word is reached
36              if i >= length:
37                  break
38
39              # One VC sequence is completed
40              m += 1
41
42              # Skip consonants
43              while i < length and self.is_consonant(word, i):
44                  i += 1
45
46          return m
47
48
49      # Apply Porter Stemmer Step 3
50      def apply_step3(self, word):
51
52          rules = {
53              "icate": "ic",
54              "ative": "",
55              "alize": "al",
56              "iciti": "ic",
57              "ical": "ic",
58              "ful": "",
59              "ness": ""
60          }
61
62          for suffix, replacement in rules.items():
63
64              if word.endswith(suffix):
65
66                  # Extract the stem
67                  stem = word[:-len(suffix)]
68
69                  # Apply the rule only when m > 0
70                  if self.measure(stem) > 0:
71                      return stem + replacement
72
73                  return word
74
75          # Return unchanged word if no suffix matches
76          return word
77
78
79  # Main Program
80  stemmer = PorterStemmerStep3()
81
82  word = input("Enter a word: ").lower()
83
84  result = stemmer.apply_step3(word)
85
86  print("Input Word :", word)
87  print("Output Word:", result)
```

Sample Output









Conclusion on Limitations

Porter Stemmer Step 3 provides a simple and computationally efficient method for reducing selected derivational suffixes. However, it is most effective when executed in the correct sequence with the remaining Porter Stemmer rules.